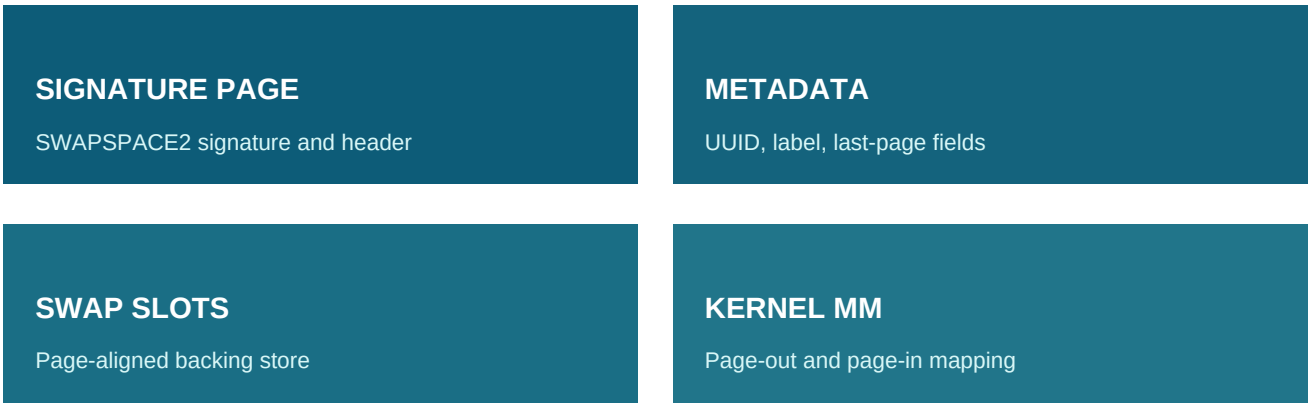
Linux swap-space disk format and history

OFFICIAL FORMAT / CHRONOLOGY / ENGINEERING

A page-oriented operating-system backing store, not a normal directory-bearing persistent file system: Linux swap areas have their own header, extents and activation rules.

This guide documents the original format's technical architecture, history and independent source references. InfiltratorOS modules and install packages are implementation artefacts, not the authority for the historic on-disk definition.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

Linux swap-space disk format and history

HISTORICAL CHRONOLOGY

Virtual memory and swapping predate Linux, originating in early time-sharing operating systems that used secondary storage for memory pressure. Linux inherited the principle of a separate swap partition or file but developed its own on-disk headers and page-allocation rules. The current Linux swap-area signature is historically distinct from both Unix filesystems and the storage of hibernation images.

The Linux swap format evolved from older versions to the SWAPSPACE2 scheme as memory capacities and kernel paging changed during the 1990s. The util-linux mkswap utility and Linux kernel memory manager document current behaviour. The history should not be described as if swap had directories, inode tables or ordinary files.

Linux swap-space disk format and history

ORIGINAL ON-DISK ARCHITECTURE

A swap area contains a header page with identifying information, usable page count, UUID, label and a signature at the end of the signature page. Most of the area consists of page-size slots available for swapping. The format is interpreted relative to the running kernel’s architecture and page-size assumptions, not as a traditional filesystem superblock with directory roots.

A swap file resides inside a separate host filesystem and is subject to alignment, stable mapping and holes or copy-on-write restrictions. A swap partition exposes contiguous block-device sectors but is not a navigable file namespace. Hibernate or resume metadata may use the same physical area under separate rules and must not be overwritten as though it were unused swap.

Linux swap-space disk format and history

ALLOCATION AND DIRECTORIES

The kernel tracks swap slot allocation in memory as virtual-memory pages are moved to and from backing storage. The on-disk area does not maintain a persistent FAT or B-tree directory mapping human filenames to data. Active swapping can make arbitrary raw reads inconsistent because slots are reused rapidly.

The system supports multiple swap areas with priorities and may choose among them based on configured policy. Swap file extents can change if their host filesystem rewrites blocks, which is why filesystem and kernel constraints on swapfiles matter. No object-per-file allocation API belongs to the Linux swap format.

Linux swap-space disk format and history

INTEGRITY AND RECOVERY

Swap is transient backing store, not a persistent user-data recovery format. The normal failure response to lost anonymous pages may be a process failure or system problem, not a filesystem repair scan. The header and signature can prove that a device is configured as swap, but not guarantee that historical process pages are meaningful after reboot.

mkswap changes the area header and can make previous content inaccessible or unusable. swapon activates it under kernel policy. Writing to an active swap device as though it were an unmounted filesystem can cause immediate memory corruption. An offline verifier should identify page geometry, headers and signatures without pretending that a conventional fsck can reconstruct directories.

Linux swap-space disk format and history

DESIGN RATIONALE AND LIMITATIONS

Swap is deliberately simple, aligned to the VM system’s allocation unit and usable by the kernel without filesystem namespace overhead. Page-oriented allocation fits virtual-memory semantics, but this simplicity makes it unlike the 22 other persistent on-media filesystem formats in the InfiltratorOS catalogue.

Swap does not implement POSIX files, journalled directory transactions, permissions per swapped page, snapshots or metadata trees. Security handling may involve discard, encryption or hibernation policy outside the swap header format. Capacity limits vary with kernel and architecture.

Linux swap-space disk format and history

INFILTRATOROS INTEGRATION BOUNDARY

SWAP.so can at most expose swap format identification and appropriate host-side capabilities. Actual paging remains a responsibility of the operating-system memory manager. The absence of a SWAP.ko under our distribution rules must not be misrepresented as a missing independent filesystem driver.

The Linux kernel’s built-in swap subsystem is not replaceable by merely placing a userspace shared library in /System/Filesystems. The manual intentionally records this fundamental distinction while describing the on-media header and historical evolution.

Linux swap-space disk format and history

PRIMARY SOURCE REFERENCES

These primary technical documents and near-contemporary references provide provenance for the design history and on-disk structures. For exact field sizes or a particular format revision, consult the original specifications and source code, not the conceptual cover illustration.

1. Linux man-pages: mkswap(8), on-disk header and limits

https://man7.org/linux/man-pages/man8/mkswap.8.html

2. util-linux mkswap source and signature handling

https://sources.debian.org/src/util-linux/2.42.2-1/disk-utils/mkswap.c

Our independent .so, optional .ko, build procedures and test evidence describe a separate implementation. The source-level history in this PDF cannot itself qualify production writes, boot, recovery or cross-platform compatibility.

Swap / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Linux swap space is a memory-management backing store, not a general-purpose filesystem with a directory tree and ordinary file records.

2. Read the swap header at the page-size-dependent location and verify the expected version and SWAPSPACE2 signature.

3. Interpret the header’s last-page boundary, bad-page inventory, UUID and optional label where the selected format revision defines them.

4. The remainder of a swap area is organised as page-sized slots managed by the kernel’s memory subsystem, not FAT clusters or Unix inodes.

5. Translate page indexes using the expected swap page size and device bounds. A header signature does not imply that every slot contains useful data.

6. Distinguish a dedicated swap partition from swap within a regular filesystem file, where the hosting filesystem controls physical extent placement.

7. Understand that transient swap pages are replaced and reused as processes run. A slot’s raw contents do not identify its owner without external memory-management state.

8. Hibernation images and resume metadata have separate conventions that must not be confused with ordinary swap-space allocation.

Swap / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Never mount or attempt directory repair on swap space as though it were ext, FAT or another filesystem; there is no such namespace to reconstruct.

2. Check header signatures, version and last-page bounds before deciding the area is usable, and exclude known bad-page slots.

3. Treat UUID and label fields as identification aids rather than integrity checksums or proof that page data is consistent.

4. Do not infer stored process identities or allocation order solely from raw swap slots; live mappings are held in operating-system memory.

5. A running system may change swap pages during acquisition. Forensic work requires a coherent image and an understanding of process and kernel state.

6. Recognise that encrypted swap may contain ciphertext and ephemeral keys can make prior contents unrecoverable.

7. Keep mkswap initialization, swapon activation, hibernation resume and secure erasure as distinct operations with different safety implications.

8. Historical virtual-memory swapping predates Linux; document the Linux on-disk swap format rather than presenting paging itself as a novel filesystem invention.